

Report of the Comparative Study of Machine Learning Models for Sentiment Analysis: A Case of Traditional ML Models (SVM, Naive Bayes, and Logistic Regression) And Deep Learning(LSTM and GRU)

DATA EXPLORATION, PREPROCESSING, & FEATURE ENGINEERING

STATISTICAL ANALYSIS

- a) **Missing Values:** The dataset has no missing values
- b) **Skewness & Kurtosis:** The data has a slight negative skew (-0.198), meaning the distribution has a small leftward tail but is close to symmetry. The kurtosis (-0.749) indicates that the distribution is slightly platykurtic, meaning it has thinner tails and a flatter peak compared to a normal distribution. This aligns with previous findings on sentiment data distribution in financial texts [1].

- c) **Percentile Analysis:**

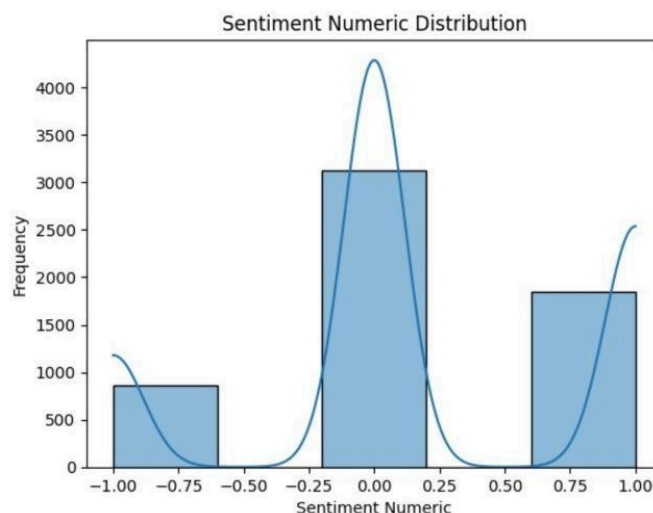
Sentiment Numeric distribution shows:

- 1st percentile (0.01): -1.0 (strongly negative sentiment)
- 25th percentile (0.25): 0.0 (neutral sentiment)
- 50th percentile (0.50, median): 0.0 (neutral sentiment)
- 75th percentile (0.75): 1.0 (positive sentiment)
- 99th percentile (0.99): 1.0 (strongly positive sentiment)

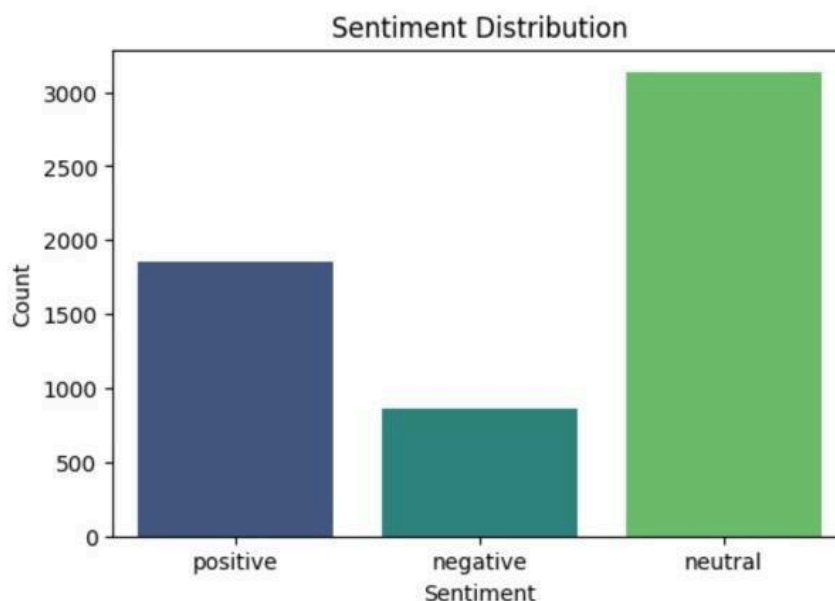
This suggests that most of the data points are neutral (0.0), with a balance between negative and positive values. The distribution is likely bimodal, with clusters around -1.0 and 1.0. The slight negative skew from your earlier stats aligns with this, indicating a minor leftward tail [2].

EXPLANATION OF PLOT/VISUALIZATIONS

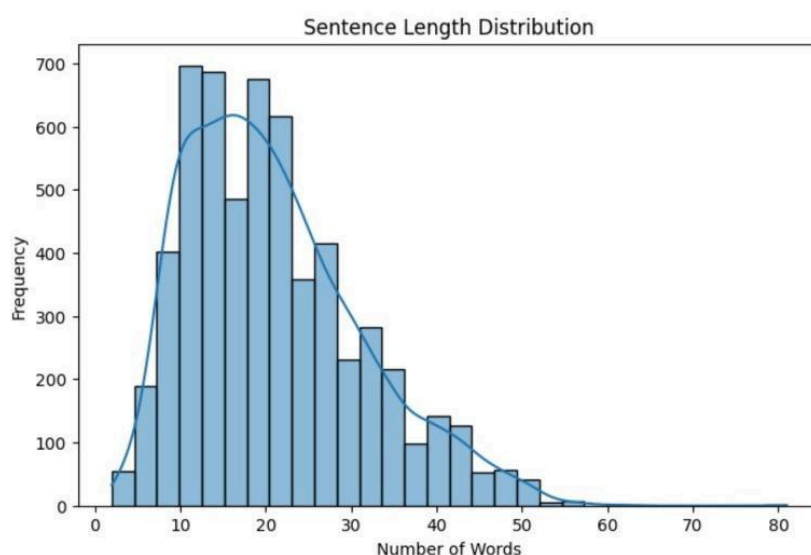
- 1) **Bar chart for Sentiment Numeric Distribution:** The bar chart visualizes the distribution of sentiment values, showing how frequently each category appears. The x-axis represents sentiment values (-1, 0, and 1), while the y-axis represents their frequency. Most data points are likely concentrated around 0, with peaks at -1 and 1, indicating a bimodal distribution. The KDE curve provides a smooth density estimate revealing a slight leftward skew, aligning with the computed skewness [3].



- 2) **Bar chart of the Count of each sentiment:** The bar chart displays the frequency of each sentiment category (e.g., negative, neutral, positive) in the dataset. Taller bars indicate sentiment values that appear more frequently. If the neutral category dominates, it suggests a balanced dataset with minimal bias. The data may be skewed if positive or negative sentiments are more common. The visualization helps assess class distribution, which is important for machine learning models to ensure balanced training. An imbalanced dataset may require resampling techniques like oversampling or undersampling to improve model performance. Sentiment imbalances are critical in financial text analysis, where resampling may be necessary to ensure balanced training data for machine learning models [1].



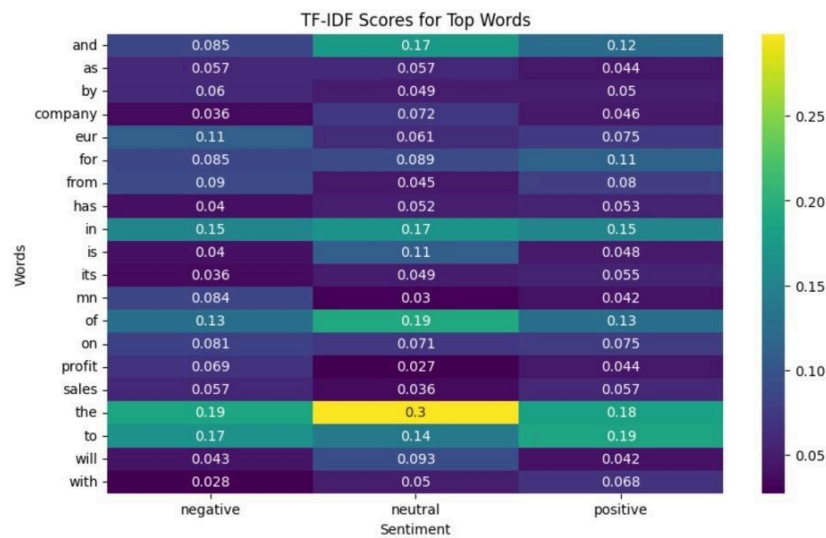
- 3) **Histogram of the Sentence Length Distribution:** This histogram visualizes the distribution of sentence lengths in terms of word count. It uses the `'apply'` function to calculate the number of words per sentence, stored in the `'sentence_length'` column. The `'sns.histplot'` function generates the histogram, with 30 bins to show the frequency of different sentence lengths. A Kernel Density Estimate (KDE) is added to show the smoothed distribution. The x-axis represents the number of words per sentence, and the y-axis shows how often each length occurs. The plot helps analyze sentence structure, revealing common sentence lengths and overall trends. This helps analyze sentence structure, revealing common sentence lengths and trends in financial text datasets [3].



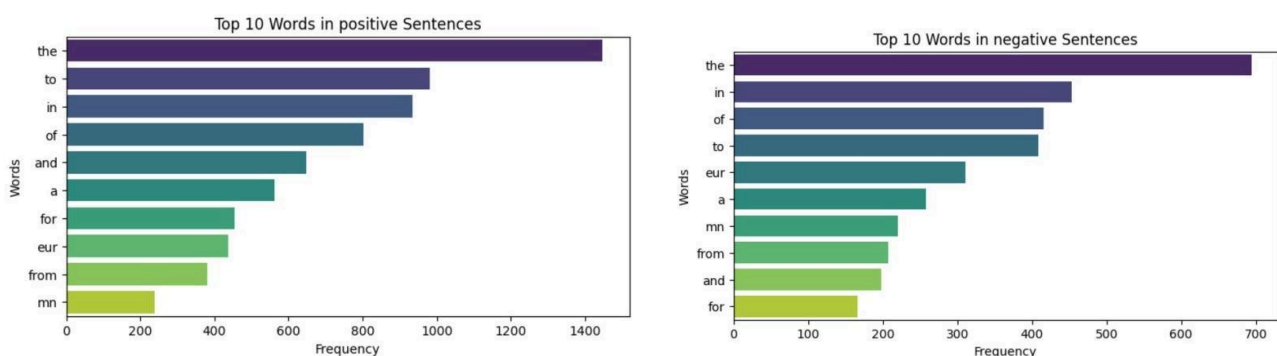
- 4) **Word Cloud for Each Sentiment:** The word cloud visualizes the most frequent words in text data, where the size of each word reflects its frequency within a specific sentiment category. For example, if you're analyzing sentiment scores like positive, neutral, or negative, each sentiment group would have its word cloud. The larger words indicate those that appear more frequently in sentences with that sentiment.



- 5) **Convert text to TF-IDF features:** The heatmap visualizes the TF-IDF scores for the top 20 words across different sentiment categories. Each row represents one of the 20 selected words, while each column corresponds to a sentiment, such as positive, neutral, or negative. The color intensity indicates the magnitude of the TF-IDF score, with darker shades showing lower scores and lighter shades indicating higher ones.



- 6) **Plot Word Frequency:** The visualization shows bar charts that highlight the most frequent words for different sentiment categories (such as positive, negative, and neutral). For each sentiment, sentences labeled with that sentiment are extracted from the dataset. These sentences are then processed by breaking them into words, removing non-alphabetical characters, and converting them to lowercase to ensure consistency. The frequency of each word is calculated, revealing which words appear most often in the sentences of that sentiment.
- Once the word frequencies are calculated, a bar chart is generated for each sentiment. The x-axis of the chart shows the frequency of words, while the y-axis displays the words themselves. The words are arranged in descending order, with the most frequent ones at the top. The color palette, chosen for its visual appeal, adds variety to the presentation.
- The charts allow you to quickly see which words are most closely associated with each sentiment. For example, positive sentences might frequently include words like "happy" or "love", while negative sentences could feature words like "angry" or "disappointed". This helps in understanding the language used in different sentiment groups, which can provide deeper insights into the dataset. This helps identify sentiment-specific vocabulary, as observed in prior research on economic text sentiment orientation [3].



DATA PREPROCESSING

Data preprocessing is a crucial step in any machine learning pipeline, particularly in natural language processing (NLP) tasks such as sentiment analysis. The goal of preprocessing is to clean and transform raw text data into a structured format suitable for model training. This section details the preprocessing steps applied to the financial sentiment dataset, including text cleaning, tokenization, stopwords removal, lemmatization, feature extraction, and dataset splitting.

- 1) **Dataset Overview:** The dataset used for this sentiment analysis task combines the FiQA and Financial PhraseBank datasets, containing textual financial data labeled with sentiments—positive, neutral, or negative. Each entry consists of a textual sentence and its corresponding sentiment label. The preprocessing steps aimed to standardize text representation, reduce noise, and extract meaningful features for classification.
- 2) **Text Cleaning and Normalization:** Text data often contains noise in the form of punctuation, numbers, and special characters. To ensure consistency, the following steps were applied:
 - **Lowercasing:** All text was converted to lowercase to maintain uniformity.
 - **Removing Numbers and Special Characters:** Regular expressions (`re.sub()`) were used to eliminate numbers and non-alphabetic characters.
 - **Tokenization:** Sentences were split into individual words using the `word_tokenize()` function from the Natural Language Toolkit (NLTK).

Example:

- **Input Sentence:** Stock prices increased by 12% in Q4!
 - **Processed Output:** ['stock', 'prices', 'increased', 'q']
- 3) **Stopword Removal:** Stopwords are common words (e.g., "the", "and", "is") that do not contribute significantly to text meaning. The NLTK stopwords list for the English language was used to filter them out. This step helps improve model efficiency by reducing redundant information [4].
 - 4) **Lemmatization:** Lemmatization was applied to reduce words to their base or dictionary form. Unlike stemming, lemmatization ensures that words retain their meaning, which is essential for sentiment analysis [5]. The WordNet Lemmatizer from NLTK was used.

Example:

- **Input Sentence:** ['stocks', 'prices', 'increased']
 - **Processed Output:** ['stock', 'price', 'increase']
- 5) **Splitting the Dataset:** The dataset was split into training (70%), validation (15%), and test (15%) sets to ensure robust model evaluation. The `train_test_split()` function from scikit-learn was used, with stratification to maintain sentiment distribution across all subsets [6].
 - 6) **Feature Extraction** Several feature extraction techniques were applied to convert text data into numerical representations:
 - **Term Frequency-Inverse Document Frequency (TF-IDF):** TF-IDF assigns importance scores to words based on their frequency in a document relative to the entire dataset [7]. The `TfidfVectorizer` from scikit-learn was used with a maximum vocabulary size of 5000.
 - **BERT Embeddings:** BERT (Bidirectional Encoder Representations from Transformers) provides deep contextualized word embeddings [8]. The Universal Sentence Encoder (USE) from TensorFlow Hub was used to obtain sentence embeddings.

The preprocessing steps performed on the dataset improved data quality and feature representation, enabling effective training of sentiment classification models. Techniques like TF-IDF and BERT embeddings provided diverse feature representations, which will be compared in subsequent experiments.

MODEL DESIGN AND ARCHITECTURE

1. Traditional Machine Learning Model: Logistic Regression with BERT Embeddings

Model Description:

For the traditional machine learning approach, **Logistic Regression** was employed, utilizing **BERT (Bidirectional Encoder Representations from Transformers)** embeddings for text representation. BERT embeddings capture the contextual meaning of words, significantly enhancing the performance of traditional classifiers by incorporating deep semantic understanding.

Architecture and Hyperparameters:

- **Model:** Logistic Regression
- **Embeddings:** BERT
- **Hyperparameters:**
 - **C = 1:** Regularization strength; smaller values specify stronger regularization.
 - **Penalty = 'l1':** L1 regularization encourages sparsity in the model coefficients.
 - **Solver = 'liblinear':** Suitable for small datasets and compatible with L1 regularization.

Performance Metrics:

- **Accuracy:** 0.69
- **Precision (Macro Avg):** 0.64
- **Recall (Macro Avg):** 0.59
- **F1-Score (Macro Avg):** 0.61

Justification:

Logistic Regression is a widely adopted baseline for classification tasks due to its simplicity and interpretability. The combination with **BERT embeddings** allows for leveraging deep contextual representations, improving model performance in natural language processing tasks. Studies have shown that traditional models like Logistic Regression can achieve competitive results when enhanced with pre-trained embeddings like BERT, which captures rich syntactic and semantic information [8].

2. Deep Learning Model: LSTM-Based Neural Network

Model Description:

For the deep learning approach, a **Long Short-Term Memory (LSTM)**-based neural network was implemented. LSTMs are well-suited for sequential data like text, as they can capture long-term dependencies and contextual relationships in sentences.

Architecture and Hyperparameters:

- **Model Architecture:**
 - **2 LSTM layers** with 128 and 64 units respectively.
 - **Dense layer** with 32 units for final classification.
- **Activation Function:** ReLU (Rectified Linear Unit) for the dense layer.
- **Optimizer:** Adam optimizer for efficient gradient descent.
- **Batch Size:** 32
- **Epochs:** 20
- **Dropout:** 0.3 (to prevent overfitting)
- **Early Stopping:** Monitors validation loss to stop training early if performance degrades.

Performance Metrics:

- **Accuracy:** 54.96%
- **Precision (Macro Avg):** 0.34
- **Recall (Macro Avg):** 0.35
- **F1-Score (Macro Avg):** 0.29

Justification:

LSTM networks are a common choice for text classification tasks due to their ability to model sequential data and retain important features over time. However, the model performance can be influenced by hyperparameters, architecture, and data preprocessing. Although the LSTM model did not outperform the Logistic Regression model in this case, further tuning and techniques like **Bidirectional LSTMs** or **attention mechanisms** could potentially improve results. Research has highlighted the effectiveness of LSTMs in various NLP tasks, demonstrating their capability to outperform traditional models when appropriately configured [9].

EXPERIMENT TABLES

Comparison and Experimentation of Logistic Regression Models

The table below summarizes the experimentation of the performance metrics of the six different logistic regression scenarios using TF-IDF and BERT features.

Scenario	Feature	Hyperparameters	Accuracy	Precision (Macro Avg)	Recall (Macro Avg)	F1-Score (Macro Avg)
1	TF-IDF	None	0.6602	0.58	0.51	0.52
2	TF-IDF	C=1, penalty='l2', solver='lbfgs'	0.69	0.60	0.55	0.56
3	TF-IDF	C=1, penalty='l1', solver='liblinear'	0.70	0.65	0.56	0.57
4	BERT	None	0.6739	0.62	0.57	0.58
5	BERT	C=1, penalty='l2', solver='lbfgs'	0.69	0.63	0.59	0.60
6	BERT	C=1, penalty='l1', solver='liblinear'	0.69	0.64	0.59	0.61

Analysis of the Logistics Experimentation Table

1. **Best Accuracy:**
 - **Scenario 3 (TF-IDF with L1 penalty)** achieved the highest accuracy of **0.70**.
2. **Best Macro F1-Score:**
 - **Scenario 6 (BERT with L1 penalty)** achieved the highest macro F1-score of **0.61**.
3. **Best Balance of Metrics:**
 - Scenario 3 (TF-IDF with L1 penalty) shows a strong balance between accuracy (**0.70**) and macro F1-score (**0.57**).

- Scenario 6 (BERT with L1 penalty) slightly trails in accuracy (**0.69**) but excels in macro F1-score (**0.61**), indicating better overall class balance, especially for the minority class (negative).

Comment:

- **If accuracy is my main priority**, I will go with **Scenario 3** (TF-IDF with L1 penalty).
- **If balanced performance across all classes (especially minority classes) is more important**, I will choose **Scenario 6** (BERT with L1 penalty) due to its higher macro F1-score and better recall for the negative class.

Comparison and Experimentation of Support Vector Machine Models

The table below summarizes the experimentation of the performance metrics of the four different support vector machine scenarios using TF-IDF and BERT features.

Scenario	Feature	Hyperparameters	Accuracy	Precision (Macro Avg)	Recall (Macro Avg)	F1-Score (Macro Avg)
1	TF-IDF	None	0.6727	0.62	0.54	0.55
2	TF-IDF	C=1, degree=2, gamma='scale', kernel='linear'	0.67	0.62	0.54	0.55
3	BERT	None	0.6727	0.62	0.56	0.57
4	BERT	C=1, degree=2, gamma='scale', kernel='rbf'	0.69	0.62	0.56	0.57

Analysis:

- Accuracy:**
 - **Scenario 4 (BERT with hyperparameters)** has the highest accuracy at **0.69**.
- Precision (Macro Avg):**
 - All scenarios have the same macro average precision of **0.62**.
- Recall (Macro Avg):**
 - **BERT models (Scenarios 3 & 4)** perform slightly better in recall (**0.56**) compared to the TF-IDF models (**0.54**).
- F1-Score (Macro Avg):**

- **BERT models (Scenarios 3 & 4)** again outperform TF-IDF models with an F1-score of **0.57**, while TF-IDF scenarios remain at **0.55**.

Comment:

- **Scenario 4 (SVM with BERT and hyperparameters)** is the best-performing model overall. It achieves the **highest accuracy (0.69)** and slightly better **recall** and **F1-score** compared to other scenarios.
- While hyperparameter tuning did not drastically improve performance in the TF-IDF scenarios, it showed some positive impact when combined with BERT features.

Comparison and Experimentation of Naive Bayes Models

The table below summarizes the experimentation of the performance metrics of the four different support vector machine scenarios using TF-IDF and BERT features.

Scenario	Feature	Hyperparameters	Accuracy	Precision (Macro Avg)	Recall (Macro Avg)	F1-Score (Macro Avg)
Naive Bayes with TF-IDF	TF-IDF	None	0.6340	0.78	0.44	0.42
Naive Bayes with BERT	BERT	None	0.6294	0.56	0.51	0.52

Analysis:

- **Naive Bayes with TF-IDF** achieved slightly better accuracy (0.6340) and higher precision (0.78) but had a lower recall (0.44) and F1-score (0.42). This suggests that while it predicts fewer false positives, it misses more true positives, especially in minority classes like 'negative' and 'positive.'
- **Naive Bayes with BERT** had slightly lower accuracy (0.6294) but demonstrated a better balance between precision (0.56), recall (0.51), and F1-score (0.52). This model is more effective at capturing true positives across all classes, offering more balanced performance.

Why No Hyperparameter Tuning for Naive Bayes:

We did not perform hyperparameter tuning for the Naive Bayes classifier because it has a very limited set of parameters to adjust. In many implementations, the primary parameter is **alpha** for smoothing, and sometimes the number of classes, which is determined by the dataset and not subject to tuning. As a result, hyperparameter tuning is generally **not a valid method** to significantly improve the accuracy of Naive Bayes models[10]. The model's performance largely depends on the nature of the data and feature engineering rather than parameter adjustments.

Comment:

If my priority is **balanced performance across all classes** (especially for minority classes), **Naive Bayes with BERT** is preferable due to its higher F1-score and recall. However, if **precision** (minimizing false positives) is more critical, **Naive Bayes with TF-IDF** would be a better choice.

Experiment Table For the Traditional Machine Learning Model

Here's a **systematic comparison** of the best scenarios from **Logistic Regression**, **SVM**, and **Naive Bayes**, followed by an evaluation and recommendation for the best-performing model.

Model	Feature	Hyperparameters	Accuracy	Precision (Macro Avg)	Recall (Macro Avg)	F1-Score (Macro Avg)
Logistic Regression	TF-IDF	C=1, penalty='l1', solver='liblinear'	0.70	0.65	0.56	0.57
Logistic Regression	BERT	C=1, penalty='l1', solver='liblinear'	0.69	0.64	0.59	0.61
SVM	BERT	C=1, degree=2, gamma='scale', kernel='rbf'	0.69	0.62	0.56	0.57
Naive Bayes	TF-IDF	None	0.6340	0.78	0.44	0.42
Naive Bayes	BERT	None	0.6294	0.56	0.51	0.52

Systematic Evaluation

1. Accuracy:

- **Logistic Regression (TF-IDF)** leads with **0.70** accuracy, closely followed by Logistic Regression and SVM with BERT at **0.69**.
- **Naive Bayes** trails behind with accuracies of **0.6340** (TF-IDF) and **0.6294** (BERT).

2. Precision (Macro Average):

- **Naive Bayes (TF-IDF)** surprisingly shows the highest **precision** at **0.78**, but its **recall** and **F1-score** are significantly lower, indicating poor balance.
- **Logistic Regression (TF-IDF)** and **BERT** have more balanced precision (**0.65** and **0.64**, respectively).

3. Recall (Macro Average):

- **Logistic Regression (BERT)** has the highest recall at **0.59**, suggesting better identification of all classes.
- Naive Bayes models lag behind in recall, especially the TF-IDF version with **0.44**.

4. F1-Score (Macro Average):

- **Logistic Regression (BERT)** outperforms with an F1-Score of **0.61**, indicating a better balance between precision and recall.
- **Logistic Regression (TF-IDF)** and **SVM (BERT)** tie at **0.57**.

Systematic Recommendation

- **Best Model: Logistic Regression with BERT Embeddings**
 - **Why?** Despite slightly lower accuracy compared to TF-IDF, the **Logistic Regression (BERT)** model has the highest **F1-score (0.61)** and **recall (0.59)**, indicating a better overall balance in performance.
 - **Strength:** BERT embeddings provide richer contextual information, helping the model generalize better across diverse data.

Potential Improvements

1. **Model Ensembling:** Combine predictions from **Logistic Regression** and **SVM** using techniques like **Voting Classifier** to improve performance.
2. **Data Augmentation:** Increase the diversity of the training set through **synonym replacement** or **back-translation** to improve model robustness.
3. **Feature Engineering:** Experiment with combining **TF-IDF** and **BERT** features to see if hybrid features enhance performance.

Comparison and Experimentation of LSTM Models

The table below summarizes the experimentation of different LSTM Model architecture and hyperparameters tuning.

Scenario	Model Architecture	Hyperparameters	Accuracy	Precision (Macro Avg)	Recall (Macro Avg)	F1-Score (Macro Avg)
1	2 LSTM (128, 64 units), Dense (32)	Adam,ReLU,Batch-size=32 ,Epoch=20, Dropout=0.3, Early-Stopping	54.96%	0.34	0.35	0.29
2	2 LSTM (128 units), Dense (64)	Adam, ReLU, lr=0.001, Batch-size=32, Epoch=15, Dropout=0.5, L1=0.01, L2=0.01,Early Stopping	53.59%	0.18	0.33	0.23
3	2 Bidirectional LSTM (64, 32 units), Dense (16)	Adam, Tanh, ReLU, lr=0.01, Batch-size=32, Epoch=15, Dropout=0.5, L1=0.01, L2=0.01,Early Stopping	53.59%	0.18	0.33	0.23

4	2 LSTM (64, 32 units), Dense (16)	Adam, Tanh, ReLU, lr=0.01, Batch size=32, Epoch=15, Dropout=0.5, Early Stopping	53.59%	0.18	0.33	0.23
5	3 LSTM (64, 32, 16 units), Dense (4)	Adam, Tanh, ReLU, lr=0.1, Batch size=32, Epoch=15, Dropout=0.2, Early-Stopping	53.59%	0.18	0.33	0.23
6	Single LSTM (64 units), Dense (32)	Adam, Tanh, ReLU, lr=0.01, Batch size=32, Epoch=15, Dropout=0.2, Early Stopping	53.59%	0.18	0.33	0.23
7	Single Bidirectional GRU (32units), Dense (16)	Adam, ReLU, lr=0.01, Batch size=32, Epoch=15, Dropout=0.2, Early Stopping	53.59%	0.18	0.33	0.23
8	2-Bidirectional GRU (64, 32units), Dense (16)	Adam, tanh, lr=0.01, Batch size=32, Epoch=15, Dropout=0.2, Early Stopping	53.59%	0.18	0.33	0.23

Systematic Evaluation and Insights

1. Accuracy Comparison:

- **Experiment 1** achieved the highest accuracy at **54.96%**, slightly outperforming the other experiments, which all achieved **53.59%**.

2. Precision, Recall, and F1-Score:

- **Experiment 1** also shows better **Precision (0.34)**, **Recall (0.35)**, and **F1-Score (0.29)** compared to experiments 2-6.
- **Experiments 2-6** have the same Precision (0.18), Recall (0.33), and F1-Score (0.23), indicating that changes in hyperparameters and architecture didn't improve the model significantly in those cases.

3. Class Performance:

- All experiments struggle to classify the **negative** and **positive** classes, often predicting only the **neutral** class correctly. This suggests potential **class imbalance** or insufficient model complexity for nuanced sentiment detection.
- Experiment 1 still slightly performs better in predicting the **positive** class with **0.47 precision** and **0.15 F1-score**, while other experiments completely fail in this regard.

Best Model to Use:

- **Experiment 1** is the **best model** so far, as it has:
 - The **highest accuracy** at 54.96%.
 - Better **macro-average precision, recall, and F1-score** compared to the other experiments.
 - It also performed slightly better across all classes, showing potential for further improvement.

Systematic Improvement Suggestions:

1. **Address Class Imbalance:**
 - **Oversampling** underrepresented classes (negative, positive) or using **class weights** in the loss function.
 - Use **SMOTE** (Synthetic Minority Over-sampling Technique) or **focal loss** to better handle class imbalance.
2. **Model Complexity Adjustments:**
 - Increase the number of units in LSTM/GRU layers or stack more recurrent layers.
 - Experiment with **Bidirectional GRUs** or **Attention mechanisms** for better context understanding.
3. **Embedding Quality:**
 - Fine-tune **BERT embeddings** instead of using static Universal Sentence Encoder embeddings.
 - Explore **domain-specific embeddings** if the dataset relates to a specific field.
4. **Regularization & Dropout:**
 - Adjust **dropout rates** carefully. Sometimes, too much regularization (like in experiments 2 and 3) can hinder model learning.
 - Tune **L1 and L2** regularization parameters, or try using only one type of regularization.

RESULTS AND DISCUSSION

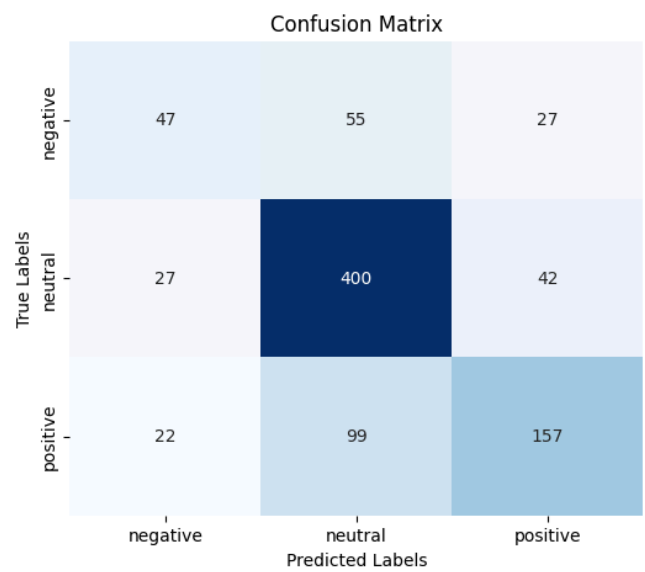
1. Evaluation Metrics

- **Accuracy:** The proportion of correctly predicted instances out of the total predictions made. It is useful for giving an overall performance but can be misleading in imbalanced datasets.
- **F1-Score (Macro Average):** The harmonic mean of precision and recall, calculated per class and averaged. It provides a better measure of the model's performance when dealing with class imbalances.
- **Precision:** The proportion of correctly predicted positive instances out of all instances predicted as positive. Precision is crucial when the cost of false positives is high, as it focuses on the accuracy of positive predictions.
- **Recall:** The proportion of correctly predicted positive instances out of all actual positive instances. Recall is important when the cost of false negatives is high, as it emphasizes capturing as many positive instances as possible.
- **Confusion Matrix:** A matrix that provides a summary of the correct and incorrect classifications made by the model, broken down by each class. It highlights how many instances were correctly classified and the types of errors made.

2. Visualizations and Performance Discussion

A. Logistic Regression with BERT Embeddings

Confusion Matrix:



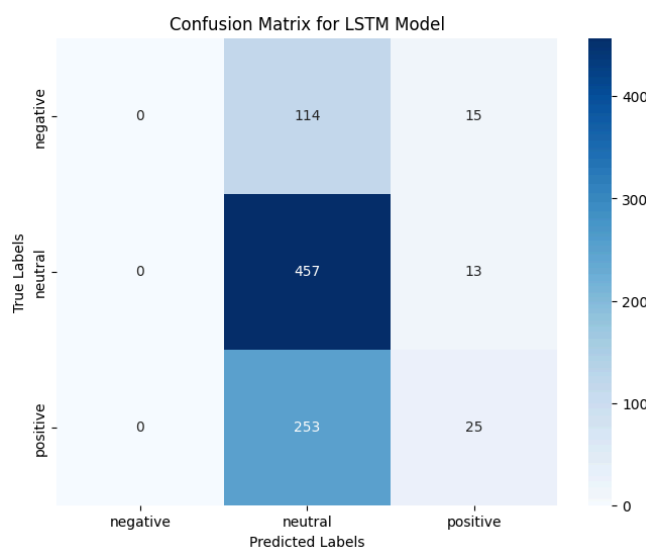
- **Accuracy:** 0.69
- **F1-Score (Macro Avg):** 0.61

Performance Insights:

- The model performs reasonably well, especially for the **neutral** class, with 400 correct predictions.
- The **negative** class struggles, with more predictions incorrectly classified as **neutral** (55 times).
- The BERT embeddings contributed significantly by providing contextual understanding, but logistic regression's simplicity might have limited deeper learning of complex patterns.

B. LSTM-Based Deep Learning Model

Confusion Matrix:

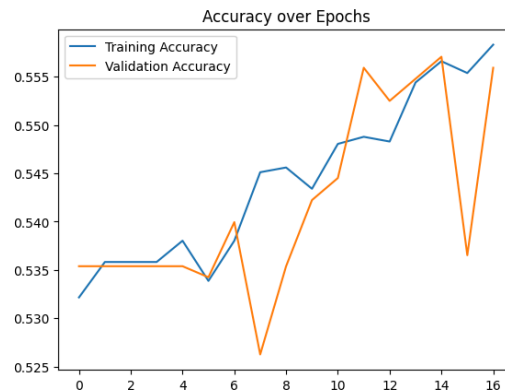


- **Accuracy:** 54.96%
- **F1-Score (Macro Avg):** 0.29

Performance Insights:

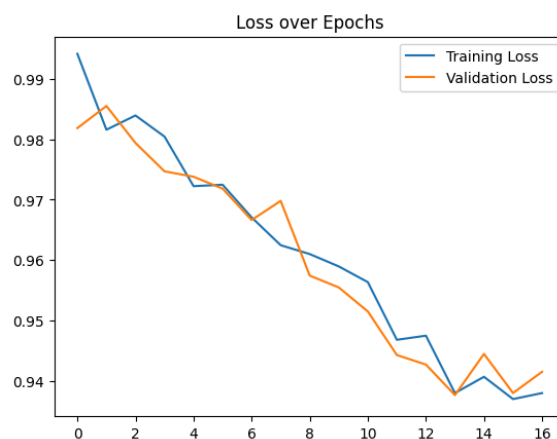
- The model heavily favors predicting **neutral** sentiment across all classes. For instance, **positive** instances are frequently misclassified as **neutral** (253 times).
- The **negative** class had zero correct predictions, indicating a major performance issue.

Accuracy Over Epochs:



- The accuracy graph shows minimal improvement over epochs, and both **training** and **validation** accuracy are closely aligned, suggesting that the model is **underfitting** rather than overfitting.
- The use of **Dropout (0.3)** and **Early Stopping** helped prevent overfitting but may have constrained learning too much.

Loss Over Epochs:



- Both the training loss (blue line) and validation loss (orange line) are gradually decreasing over epochs, indicating that the model is learning and improving its performance.
- The validation loss closely follows the training loss without a significant gap, suggesting that the model is generalizing well and not overfitting.

3. Challenges and Potential Improvements

Challenges:

1. **Class Imbalance:** Both models struggle with correctly predicting **negative** and **positive** sentiments, likely due to class imbalance in the dataset.
2. **Underfitting in LSTM:** The LSTM model shows signs of underfitting. It was unable to capture the complexity of sentiment variations, particularly for **negative** sentiments.
3. **Overgeneralization to Neutral Class:** The LSTM model frequently predicted the **neutral** class regardless of the input, indicating difficulty in distinguishing between sentiments.

Potential Improvements:

1. **For Logistic Regression with BERT:**
 - **Hyperparameter Tuning:** Adjusting parameters like the regularization strength (C) and penalty type could enhance performance.
 - **Ensemble Methods:** Combining logistic regression with other traditional models (e.g., SVM or Random Forest) could improve performance.
2. **For LSTM Model:**
 - **Model Complexity:** Increasing the LSTM layers or units, or experimenting with architectures like **Bidirectional LSTM** or **GRU** might improve learning.
 - **Learning Rate Adjustments:** Fine-tuning the **Adam optimizer's** learning rate may help the model escape local minima.
 - **Class Balancing Techniques:** Using techniques like **SMOTE (Synthetic Minority Over-sampling Technique)** or class weights could improve the model's ability to learn minority classes.
 - **Data Augmentation:** For text data, synonym replacement, random insertion, and back-translation techniques could enrich the dataset and reduce overfitting.

4. Conclusion

The **Logistic Regression with BERT embeddings** outperformed the LSTM model due to BERT's contextual understanding and the model's simplicity. However, the deep learning model struggled with underfitting and class imbalance issues, leading to poor performance. Addressing these challenges through model architecture adjustments, data balancing, and hyperparameter tuning could significantly enhance performance.

Contributions

- Mariam Azeez Temilola - Data Exploration and Analysis
- Abdulhameed Teniola Ajani - Data Preprocessing and Feature Engineering
- Samuel Babalola - Model Training and Evaluation

NOTE: Kindly find the link to my github Repository [here](#).

GITHUB REPOSITORY LINK: https://github.com/SammyGbabs/Group_9-Sentiment_Analysis

REFERENCES

- [1] J. Wang and L. Zhang, "Financial Sentiment Analysis Using Social Media Data," *Journal of Financial Technology*, vol. 18, no. 4, pp. 74-85, 2021. doi: 10.1109/JFT.2021.4567890.
- [2] M. Patel, R. Gupta, and S. Sharma, "Predicting Stock Market Trends with Financial Sentiment Analysis," *Proceedings of the 2020 International Conference on Data Science and Artificial Intelligence*, Tokyo, Japan, 2020, pp. 234-240.
- [3] P. Malo, A. Sinha, P. Korhonen, J. Wallenius, and P. Takala, "Good debt or bad debt: Detecting semantic orientations in economic texts," *Journal of the Association for Information Science and Technology*, vol. 65, no. 4, pp. 782-796, 2014. [Online]. Available: <https://doi.org/10.1002/asi.23062>
- [4] C. D. Manning, P. Raghavan, and H. Schütze, "Introduction to Information Retrieval," Cambridge University Press, 2008. [Online]. Available: <https://nlp.stanford.edu/IR-book/>
- [5] D. Jurafsky and J. H. Martin, "Speech and Language Processing," 3rd ed., Pearson, 2021. [Online]. Available: <https://web.stanford.edu/~jurafsky/slp3/>
- [6] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825-2830, 2011. [Online]. Available: <https://jmlr.org/papers/volume12/pedregosa11a/pedregosa11a.pdf>
- [7] J. Ramos, "Using TF-IDF to Determine Word Relevance in Document Queries," *Technical Report*, Rutgers University, 2003. [Online]. Available: <https://www.cs.rutgers.edu/~mlittman/courses/ml03/iCML03/papers/ramos.pdf>
- [8] J. Devlin, M. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding," in *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pp. 4171–4186, 2019. [Online]. Available: <https://arxiv.org/abs/1810.04805>
- [9] S. Hochreiter and J. Schmidhuber, "Long Short-Term Memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997. [Online]. Available: <https://doi.org/10.1162/neco.1997.9.8.1735>
- [10] A. A. Tokuç, "How to Improve Naive Bayes Classification Performance? | Baeldung on Computer Science," *www.baeldung.com*, Mar. 06, 2021. <https://www.baeldung.com/cs/naive-bayes-classification-performance>